RAPPORT DE PROJET

Application de gestion médicale

Mustapha Mountassir

ENCADRÉ PAR :
Youness LAGHOUAOUTA

Langague :
JAVA

---

1. Introduction du Projet

Le projet consiste en la conception et la réalisation d'une Application de Gestion de Cabinet Médical développée en Java. L'objectif principal est de dématérialiser le processus de consultation, de la prise de rendez-vous jusqu'à la génération de l'ordonnance.

L'application permet de :
Gérer les acteurs : Identification des médecins (avec spécialités et tarifs) et des patients (avec historique médical).

Suivre le cycle de consultation : Planification via un système de rendez-vous et calcul automatique de la durée et du coût.

Automatiser la facturation : Génération d'une interface graphique (GUI) pour les ordonnances et export automatique sous format texte (.txt).

Sécuriser les opérations : Utilisation d'exceptions personnalisées pour gérer les erreurs de paiement ou de stock.

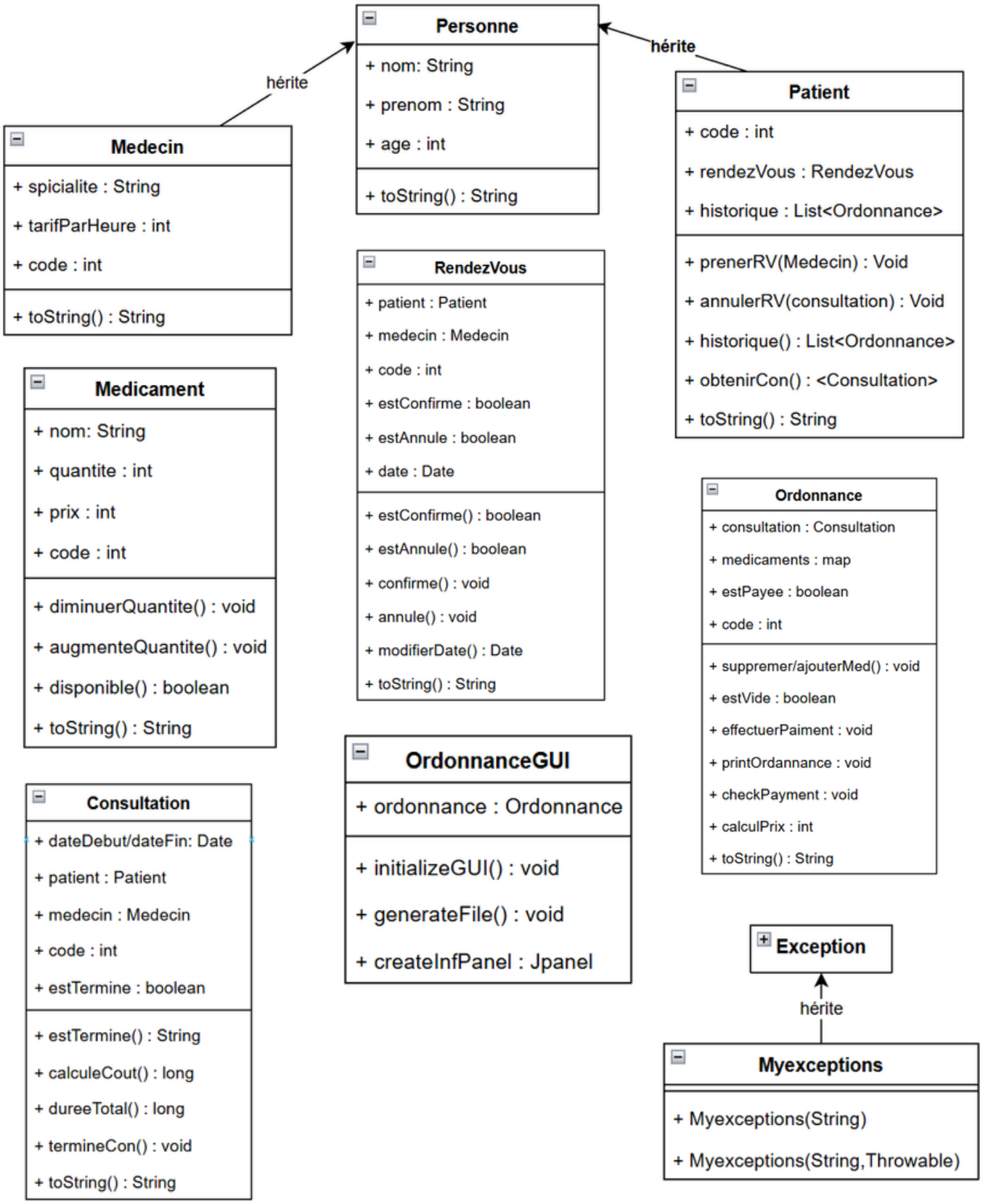
2. Scénario principal :

Le patient prend rendez-vous à une date donnée et avec le médecin de son choix. Le médecin doit ensuite valider le rendez-vous pour qu'il soit pris en compte par le système.

Chaque médecin possède un tarif horaire. Lors d'une consultation, le médecin instancie une nouvelle consultation. À la fin de celle-ci, le médecin peut prescrire des médicaments au patient ; si c'est le cas, il doit ajouter ces médicaments à l'ordonnance.

Enfin, le prix de chaque consultation (tarif selon la durée + prix des médicaments) est calculé automatiquement. L'ordonnance est ensuite générée au format .txt pour être imprimée et remise au patient.

3. Présentation d'UML :



4. Test :

la méthode main avec tous les tests sont dans GitHub (class Main)

```java
public static void main(String[] args) {
    System.out.println("=== SYSTÈME DE GESTION MÉDICALE - TEST COMPLET ===\n");
    System.out.println("--- 1. CRÉATION DES MÉDECINS ---");
    Medecin med1 = new Medecin("Mustapha", "Mountassir", 20, "Cardiologie", 5000);
    Medecin med2 = new Medecin("youness", "essabri", 20, "Pédiatrie", 100);
    Medecin med3 = new Medecin("flan", "Ahmed", 52, "Générale", 300);
    System.out.println(med1);
    System.out.println(med2);
    System.out.println(med3);
    System.out.println();
}
```

```
=== SYSTÈME DE GESTION MÉDICALE - TEST COMPLET ===

--- 1. CRÉATION DES MÉDECINS ---
Dr. Mustapha Mountassir (20 ans ) - Cardiologie : 5000 par heure;
Dr. youness essabri (20 ans ) - Pédiatrie : 100 par heure;
Dr. flan Ahmed (52 ans ) - Générale : 300 par heure;
```

Création des médecins

```java
Patient patient1 = new Patient("Tazi", "Mohammed", 35);
Patient patient2 = new Patient("Idrissi", "Amina", 28);
Patient patient3 = new Patient("Benjelloun", "Karim", 42);
```

Création des patients

```java
patient1.prenerRendezVous(med1);
patient2.prenerRendezVous(med2);
patient3.prenerRendezVous(med3);
```

Prise de rendez-vous

```java
RendezVous rv1 = patient1.getRendezVous();
RendezVous rv2 = patient2.getRendezVous();
RendezVous rv3 = patient3.getRendezVous();
System.out.println(rv1);
System.out.println(rv2);
System.out.println(rv3);
```

```
le rendez-vous sous le code #1 concern MrTazi Mohammedavec Dr. MustaphaMountassi
le rendez-vous sous le code #2 concern MrIdrissi Aminaavec Dr. younessessabri a
le rendez-vous sous le code #3 concern MrBenjelloun Karimavec Dr. flanAhmed a Sa
```

Print les rendez-vous

```java
rv1.confirme();
rv2.confirme();
rv3.annuler();
System.out.println("RV1 confirmé: " + rv1.estConfirme());
System.out.println("RV2 annuler : " + rv2.estAnnule());
System.out.println("RV3 confirme: " + rv3.estConfirme());
```

```
le rendez-vous #1 est confirme .
le rendez-vous #2 est confirme .
le rendez-vous #3 est annuler .
RV1 confirmé: true
RV2 annuler : false
RV3 confirme: false
```

Confirmation et annulation de rendez-vous

```java
Consultation consult1 = new Consultation(patient1, med1);
Consultation consult2 = new Consultation(patient2, med2);
```

Création des consultations

```java
try {
    System.out.println("Consultation en cours...");
    Thread.sleep(2000);
} catch (InterruptedException e) {
    e.printStackTrace();
}

System.out.println(consult1);
System.out.println(consult2);
System.out.println();
```

```
Consultation en cours...
 code : 1,patien : Mr. Tazi,medecin : Dr. Mustapha
 etat :en cours
 code : 2,patien : Mr. Idrissi,medecin : Dr. youness
 etat :en cours
```

Création des consultations

```java
Medicament med_doliprane = new Medicament("Doliprane", 25, 100);
Medicament med_amoxicilline = new Medicament("Amoxicilline", 45, 50);
Medicament med_paracetamol = new Medicament("Paracétamol", 15, 200);
Medicament med_ibuprofene = new Medicament("Ibuprofène", 30, 75);
Medicament med_aspegic = new Medicament("Aspégic", 20, 150);
```

Ceation des medicaments

```java
System.out.println("Doliprane disponible (30 unités): " +
        med_doliprane.verifierDisponiblilite(30));
    System.out.println("Amoxicilline disponible (60 unités): " +
        med_amoxicilline.verifierDisponiblilite(60));
    System.out.println();
```

```
Doliprane disponible (30 unités): true
Amoxicilline disponible (60 unités): false
```

Vérification de disponibilité des médicaments


```java
System.out.println("Doliprane disponible (30 unités): " +
        med_doliprane.verifierDisponiblilite(30));
    System.out.println("Amoxicilline disponible (60 unités): " +
        med_amoxicilline.verifierDisponiblilite(60));
    System.out.println();
```

```
Doliprane disponible (30 unités): true
Amoxicilline disponible (60 unités): false
```

Vérification de disponibilité des médicaments


```java
Ordonnance ord1 = new Ordonnance(consult1);
ord1.ajouterMedicament(med_doliprane, 2);
ord1.ajouterMedicament(med_amoxicilline, 3);
ord1.ajouterMedicament(med_paracetamol, 1);
```

Création des ordonnances

```java
Ordonnance ord2 = new Ordonnance(consult2);
ord2.ajouterMedicament(med_ibuprofene, 2);
ord2.ajouterMedicament(med_aspegic, 1);

System.out.println("Ordonnance 1 créée avec " +
    ord1.getMedicamentsQuantiteMap().size() + " médicaments");
System.out.println("Ordonnance 2 créée avec " +
    ord2.getMedicamentsQuantiteMap().size() + " médicaments");
System.out.println();
```

```
Ordonnance 1 créée avec 3 médicaments
Ordonnance 2 créée avec 2 médicaments
```

ajoute des medicament

```java
System.out.println("Prix total ordonnance 1: " + ord1.calculePrixTotale() + " DH");
System.out.println("Prix total ordonnance 2: " + ord2.calculePrixTotale() + " DH");
```

```
Prix total ordonnance 1: 5200 DH
Prix total ordonnance 2: 180 DH
```

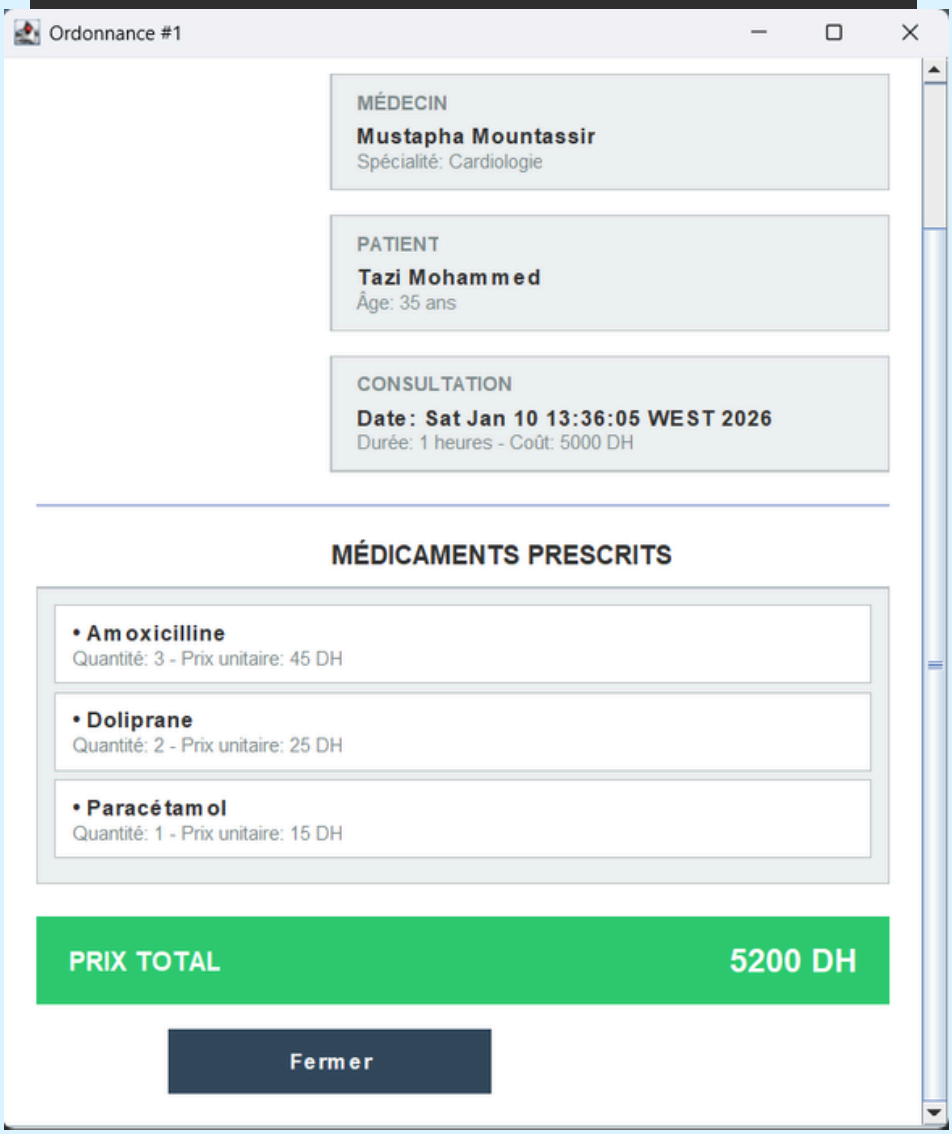
Calcul des prix

```java
ord1.effectuerPaiment(100);
```

```
Error :Montant insuffisant! requis : 5200 Dirhames, Fourni: 100 Dirhames
```

test d'exception

```java
patient1.ajouterOrdonnance(ord1);
patient2.ajouterOrdonnance(ord2);
ord1.printOrdannance();
```



```
====detailles de l'ordonnance #1====
Medecin : Mustapha Mountassir
Patient : Tazi Mohammed
Date de la consultation : Sat Jan 10 13:37:32 WEST 2026
Duree de la consultation : 1 heures
Cout de la consultation : 5000 DH
Medicaments prescrits :
        STATUS DE PAIEMENT  : false
  khlass a sga3Amoxicilline : 3 (45 DH par unité)
   Doliprane : 2 (25 DH par unité)
   Paracétamol : 1 (15 DH par unité)
       prix total : 5200 DH
```

Fichier cree avec succees

5. Use cases :

UC1 : Prendre rendez-vous

Acteur : Patient
Description : Le patient prend un rendez-vous avec un médecin disponible
Préconditions : Patient enregistré dans le système
Postconditions : Rendez-vous créé avec statut "non confirmé"

Scénario principal :

1. Le patient choisit une date
2. Le patient sélectionne un médecin
3. le système affiche les disponibilités
4. Le système crée le rendez-vous
5. Le système génère un code unique pour le rendez-vous

UC2 : Gérer consultation

Acteur : Médecin
Description : Le médecin effectue une consultation et enregistre les informations
Préconditions : Rendez-vous confirmé
Postconditions : Consultation créée avec durée et coût calculés

Scénario principal :

1. Le médecin démarre la consultation
2. Le système enregistre l'heure de début
3. Le médecin examine le patient
4. Le médecin termine la consultation et genère l'ordonnance
5. Le système calcule automatiquement la durée et le coût

UC3 : Créer ordonnance

Acteur : Médecin
Description : Création d'une ordonnance avec prescription de médicaments
Préconditions : Consultation terminée
Postconditions : Ordonnance créée avec liste de médicaments

Scénario principal :

1. Le médecin sélectionne les médicaments
2. Le médecin indique les quantités
3. Le système vérifie la disponibilité en stock
4. Le système calcule le prix total
5. Le système génère l'ordonnance

UC4 : Effectuer paiement

Acteur : Patient / Réceptionniste
Description : Paiement de l'ordonnance avec validation du montant
Préconditions : Ordonnance créée
Postconditions : Paiementvalidé, ordonnance marquée comme payée

Scénario principal :

1. Le système affiche le montant total
2. Le patient fournit le paiement
3. Le système valide le montant
4. Le système calcule la monnaie
5. Le système marque l'ordonnance comme payée

6. Le système génère le reçu

Scénario alternatif :

- Si montant insuffisant → Exception levée, paiement refusé

UC5 : Gérer stock médicaments

Acteur : Medcin
Description : Gestion des quantités de médicaments disponibles chez le medcin
Préconditions : Accès au système
Postconditions : Stock mis à jour

Scénario principal :

1. Le medcin vérifie son stock des medicaments
2. Le medcin ajoute ou retire des quantités
3. Le système met à jour les disponibilités
4. Le système signale les stocks faibles

---

6. CONCLUSION

Le système de gestion médicale développé constitue une solution complète et professionnelle pour la gestion quotidienne d'un cabinet médical. L'architecture orientée objet garantit une base solide, modulaire et évolutive.

annex :

- Voici le lien vers nos projet **Java** sur **GitHub** : https://github.com/Mount1ssir/poo2-Project

- Voici le lien vers le DEMO sur **Drive** : https://drive.google.com/drive/folders/1NB8NW_ljJubN-fqd4NkiEvig2e8pZSnr?usp=drive_link

Cordialement
Mustapha Mountassir